

Dilema del agente doble

- **Límite de tiempo:** 12 minutos.
- **Almacenamiento:** 5 GB
- **Entorno:** una GPU (≈ 16 GB VRAM), sin internet
- **Tamaño de la solución:** `solution.ipynb` ≤ 1 MB
- **Puntuación del baseline:** 0

En el centro nacional de IA de Astaná, dos modelos —el Modelo R (un ResNet-18) y el Modelo V (un ViT-Tiny)— están analizando fotografías. En este momento, ambos modelos realizan un trabajo perfecto: obtienen una exactitud del 100% y coinciden en todas y cada una de las imágenes. Para comprobar cuán diferentes son realmente sus «cerebros» inteligentes, el científico jefe le plantea un desafío: realizar cambios diminutos, casi invisibles, en los píxeles de cada fotografía para que el Modelo R y el Modelo V estén completamente en desacuerdo.



1. Tarea

Dos clasificadores de imágenes preentrenados observan la misma imagen. En las imágenes proporcionadas en esta tarea, ambos clasificadores alcanzan una exactitud del 100%.

- **Modelo R:** `torchvision.models.resnet18` (una CNN, ResNet18).
- **Modelo V:** `timmm de vit_tiny_patch16_224` (un Transformer, ViT-Tiny).

Su tarea consiste en crear un pequeño cambio («perturbación») para cada imagen de modo que los dos modelos estén en desacuerdo. Para cada imagen, debe crear **dos perturbaciones diferentes**:

- **Tipo A:** después de añadirla, el Modelo R sigue clasificando la imagen correctamente, pero el Modelo V la clasifica incorrectamente.
- **Tipo B:** después de añadirla, el Modelo V sigue clasificando la imagen correctamente, pero el Modelo R la clasifica incorrectamente.

Cada perturbación debe ser lo suficientemente *pequeña* como para que sea difícil de percibir. Las perturbaciones más pequeñas obtienen una puntuación mayor (véase la Sección 5). La perturbación se aplica directamente a la imagen original en el nivel de los píxeles.

2. Datos públicos

Con la tarea se proporciona un conjunto de imágenes, organizado en dos particiones: `train` (100 imágenes) y `test_public` (100 imágenes), cada una con imágenes de distintas resoluciones. Todas las imágenes pertenecen a las 1000 clases de ImageNet-1K, y tanto el Modelo R como el Modelo V alcanzan una exactitud del 100% en ambas particiones.

Se proporcionan los siguientes archivos:

```
train/images/*.png      # 100 images in PNG format
train/labels.json       # maps each image index to its correct class
test_public/images/*.png # 100 images in PNG format
test_public/labels.json  # maps each image index to its correct class
```

Durante la evaluación, su carpeta `dataset/test_public/` se reemplaza de forma transparente por dos conjuntos ocultos de imágenes (`test_leaderboard_a` y `test_leaderboard_b`) para la puntuación oficial. Cada uno contiene **100 imágenes** en formato PNG y un archivo de etiquetas.

Nota: Para esta tarea, las etiquetas de los datasets de prueba son accesibles.

3. Formato de salida

Para cada imagen, debe generar dos archivos:

```
{index}_a.pt  # Type A perturbation
{index}_b.pt  # Type B perturbation
```

- `{index}` (0, 1, 2, ...), coincide con el nombre de la imagen en los datasets.
- Cada archivo es un único tensor guardado con `torch.save`. Su forma debe ser $3 \times H \times W$, donde H y W coinciden con la resolución **original** de esa imagen (no 224×224).

- El código debe generar únicamente un archivo ZIP, `submission.zip`. Coloque todos los archivos `.pt` en el nivel superior del archivo ZIP, sin ninguna carpeta contenedora ni subdirectorios.

```
submission.zip
├─ 0_a.pt
├─ 0_b.pt
├─ 1_a.pt
└─ ...
```

El notebook le avisará si hay algún problema con el formato de salida.

4. Restricciones

- **Modelos:** Debe usar `torchvision.models.resnet18(pretrained=True)` y `timm.create_model('vit_tiny_patch16_224', pretrained=True)`. No se permite ningún otro modelo preentrenado.
- **Pipeline de transformaciones (aplicado obligatoriamente durante la evaluación):** `Resize(256) → CenterCrop(224) → Normalize(mean=[0.485,0.456,0.406], std=[0.229,0.224,0.225])`. See Section 3 of `baseline.ipynb` para más detalles.
- **Resolución de la perturbación:** Debe coincidir con la resolución **original** de la imagen sin procesar (no 224×224). El tensor se añade a la imagen sin procesar *antes* del pipeline de transformaciones.
- **Formato de salida:** Solo archivos `.pt`; no PNG/JPG. Los tensores se añaden a la imagen sin procesar y los valores de los píxeles se recortan a `[0, 1]` antes del preprocesamiento.
- **Nomenclatura de archivos:** Listado plano, con el formato estricto `{index}_a.pt` / `{index}_b.pt`. Sin subdirectorios dentro del archivo zip.
- **Bibliotecas:** `torch`, `torchvision`, `timm`.

5. Puntuación

La puntuación final se calcula de la siguiente manera. Sea M el número de imágenes de la partición, $Score_A$ el número de perturbaciones de Tipo A exitosas y $Score_B$ el número de perturbaciones de Tipo B exitosas:

$$S_{final} = \frac{Score_A + Score_B}{2M} \times PF$$

PF es una función diseñada para penalizar las perturbaciones con una norma alta y para ser muy sensible cerca del límite máximo de rendimiento. Esta está acotada en el intervalo de 0.5 a 1. La implementación completa puede consultarse en la Sección 8 de `solution.ipynb`.

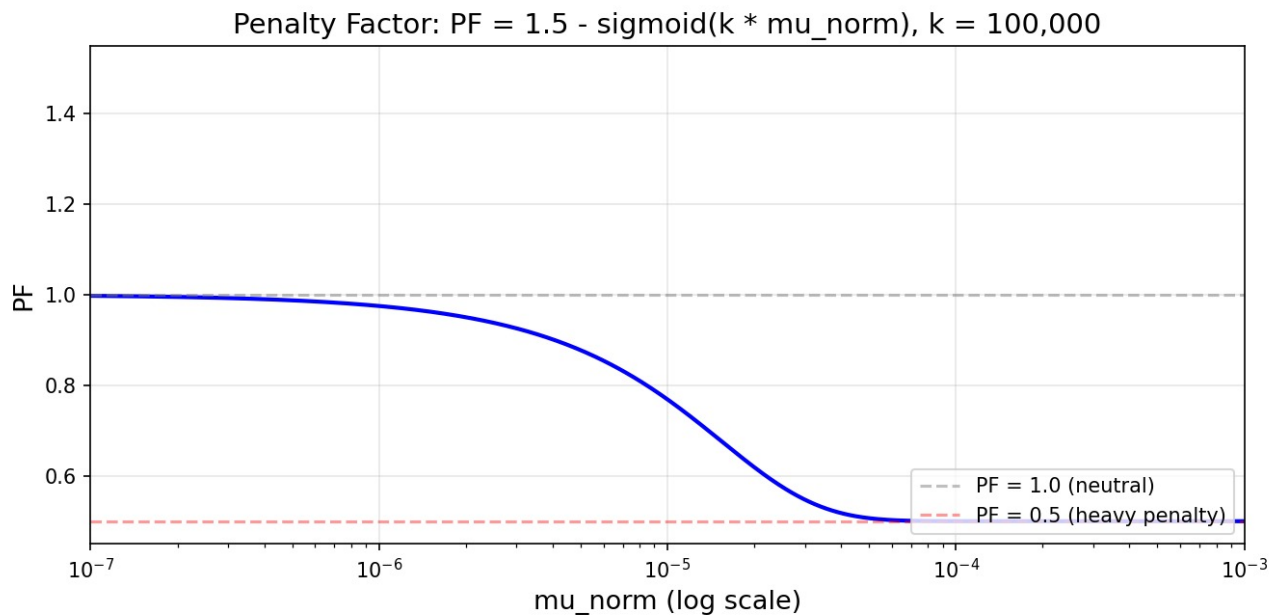


Figura: La curva de la función de penalización.

6. Comprobación del envío

El notebook contiene comprobaciones que le avisan si hay problemas de formato, en la Sección 7 del notebook `solution.ipynb`.

7. Pruebas locales

`solution.ipynb` contiene un ejemplo completo y funcional. Carga los datos públicos, ambos modelos y el evaluador oficial, y genera un archivo ZIP de envío. Léalo antes de comenzar.

8. Cómo enviar

- Guarde sus cambios en `solution.ipynb`.
- Abra la pestaña Git de la barra lateral izquierda de JupyterLab.
- Añada al **stage** `solution.ipynb` (el icono + que aparece junto a él).
- Introduzca un mensaje de commit y haga clic en **Commit**.
- Haga clic en la nube con una flecha hacia arriba para hacer push.
- Regrese a esta página del concurso y haga clic en **Submit**.

Envíe exactamente un archivo, llamado `solution.ipynb`.